PantryLens AI Usage Statement

Prepared: 8 November 2026

Project: PantryLens

Statement for milestone acceptance

AI tools were used during PantryLens development and documentation. The development models reported by the project owner were **Gemini Argon 4** and **Claude Opus 5**. The application also uses **Qwen3-VL-8B-Instruct (**`qwen/qwen3-vl-8b-instruct`**)**, accessed through **OpenRouter**, to identify pantry items in user-submitted photos. AI suggestions supported development; project decisions, code changes, and acceptance remain subject to human review.

This statement records the tools and application behavior identified from the repository and information provided by the project owner. It does not claim that AI generated every part of the project.

AI tools and models

Use	Tool or model	How it was used / is used	Record details
Development assistance	Gemini Argon 4	Used to help build PantryLens, as reported by the project owner.	Dates of use were not recorded.
Development assistance	Claude Opus 5	Used to help build PantryLens, as reported by the project owner.	Dates of use were not recorded.
In-app photo recognition	Qwen3-VL-8B-Instruct, model ID `qwen/qwen3-vl-8b-instruct`, through OpenRouter	Compares a pantry photo with the current inventory checklist and returns IDs for items it recognizes.	The app reads `OPENROUTER_MODEL`; the code default is this model ID. Actual requests use the configured model value.

How the in-app AI feature works

1. A user captures a pantry photo and explicitly selects **Identify Items**.

2. The app resizes the image so its longest side is at most 768 pixels, encodes it as JPEG at quality 72, and prepares it as a base64 data URL.

3. The app sends the image and the current checklist's item IDs and names to OpenRouter's Chat Completions endpoint (`POST https://openrouter.ai/api/v1/chat/completions`). The request includes instructions to identify only clearly visible items from that checklist.

4. The model is asked to return a strict JSON object containing `foundItemIds`. The app validates the response and rejects malformed output or IDs that are not in the current checklist.

5. If validation succeeds, matched items are marked stocked and unmatched items are marked not stocked. If the request or validation fails, the app keeps the inventory unchanged.

The image processing and request flow are implemented in `src/screens/ScanScreen.tsx`, `src/utils/imageProcessor.ts`, `src/services/openRouterService.ts`, and `src/config/env.ts`.

Data sent to the service

For an identification request, the app sends the processed pantry photo and each current inventory item's ID and name to OpenRouter. The request code does not include the inventory item's category, storage location, expiry date, notes, or checked state in the recognition checklist. The recognition call is made only after the user initiates identification.

The repository confirms the app-to-OpenRouter request, but does not establish OpenRouter's or any upstream model provider's retention, training, or deletion practices. Those practices should be checked against the applicable provider terms and account settings before making separate privacy claims.

Configuration and security note

The model can be configured with `OPENROUTER_MODEL`; if it is unset, the app defaults to `qwen/qwen3-vl-8b-instruct`. The OpenRouter API key is read from `OPENROUTER_API_KEY`. Babel inlines these values during bundling, and the app calls OpenRouter directly from the mobile client without a backend proxy. Therefore, a key included in a distributed app bundle should be treated as extractable. The `.env` file is ignored by Git; do not place a real key in this document or commit it to the repository.

Review and limitations

AI-generated development suggestions and photo recognition can be wrong. The recognition flow limits matches to existing checklist entries and validates returned IDs, but these checks do not prove that a visual identification is correct. A person remains responsible for reviewing the inventory and all milestone submissions.

This document was prepared from the repository and the project owner's account of development tool use. The dates and contribution breakdown for those sessions were not available in the project files and are left unspecified rather than inferred.